Automata and Formal Languages

Notes made by Finley Cooper

16th October 2026

Contents

1 Introduction **3**

2 Regular Languages and Finite-State Automata **3**
2.1 Deterministic Finite-State Automata 3
2.2 Regular Languages 5
2.3 Non-determinism and ε-transitions 5
2.4 Equivalence of DFAs and (ε-)NFAs 7
2.5 Regular Expressions 9
2.5.1 Regular expressions define regular languages 10

1 Introduction

In this course we will have $0 \in \mathbb{N}$.

Definition. (Alphabet) An *alphabet* Σ is a finite non-empty set with elements which are *symbols* or *letters*.

Definition. (String) A *string* or *word* of length m over Σ is $\alpha = x_1 x_2 \dots x_m$ with $x_i \in \Sigma$ or more formally $\alpha \in \Sigma^m$. We write $|\alpha| = m$ for the length of α.

Remark. We will denote the empty word as ε and denote

$$\Sigma^* = \bigcup_{n \in \mathbb{N}} \Sigma^n$$

and $\Sigma^\dagger = \Sigma^* \setminus \{\varepsilon\}$.

Definition. (Concatenation) If $\alpha = x_1 \cdots x_m \in \Sigma^m$ and $\beta = y_1 \cdots y_n \in \Sigma^n$ then $\alpha\beta$ is $x_1 \cdots x_m y_1 \cdots y_n \in \Sigma^{m+n}$.

For $0 \le i \le j \le m$, $x_{i+1} \cdots x_j$ is a substring of α of length $j - i$. So for $\alpha, \beta \in \Sigma^*$, β is a substring of α if and only if there exists $\gamma, \delta \in \Sigma^*$ with $\alpha = \gamma\beta\delta$.

Definition. (Homomorphism) For alphabets Σ, Π, a *homomorphism* $h : \Sigma^* \to \Pi^*$ satisfies $h(\alpha\beta) = h(\alpha)h(\beta)$ for all $\alpha, \beta \in \Sigma^*$.

Remark. This implies that $h(\varepsilon) = \varepsilon$.

Remark. Alternatively we can just consider a function $f : \Sigma \to \Pi^*$ which defines a unique homomorphism extending f by concatenation.

Definition. (Formal Langauge) A *formal language* L over Σ is *any* subset of Σ^*.

2 Regular Languages and Finite-State Automata

2.1 Deterministic Finite-State Automata

Remark. Suppose we want to describe a language L over Σ using a finite amount of information. We could extend Σ with other symbols to get an alphabet $\Pi \supset \Sigma$. and assign a word $w \in \Pi^*$ to L.

We can do this for countably many L. But we want w to *describe* L. We'll now see a countable class of languages where we can do this symbolically. We first give a computational method to determine these languages.

Definition. (Deterministic Finite-State Automata) A *Deterministic Finite-State Automata* (DFA) is a structure

$$D = (Q, \Sigma, \delta, q_0, F)$$

where,

(i) Q is a finite non-empty set of states;
(ii) Σ is an alphabet;
(iii) δ is a transition function from $Q \times \Sigma \to Q$;
(iv) Starting state $q_0 \in Q$;
(v) $F \subseteq Q$ is a set of acceptance states.

The idea for a DFA is that we input a word $w \in \Sigma^*$ we call $w = x_1 \cdots x_n$. The DFA starts in state q_0 and reads the first letter of the input string x_1. The DFA then transitions according to δ to $q_1 = \delta(q_0, x_1) \in Q$. We then repeat this process with x_2 and so on. Once the string is exhusted the string is accepted if $q_n \in F$ otherwise it is rejected. This defines a language.

We describe these with tables and diagrams.

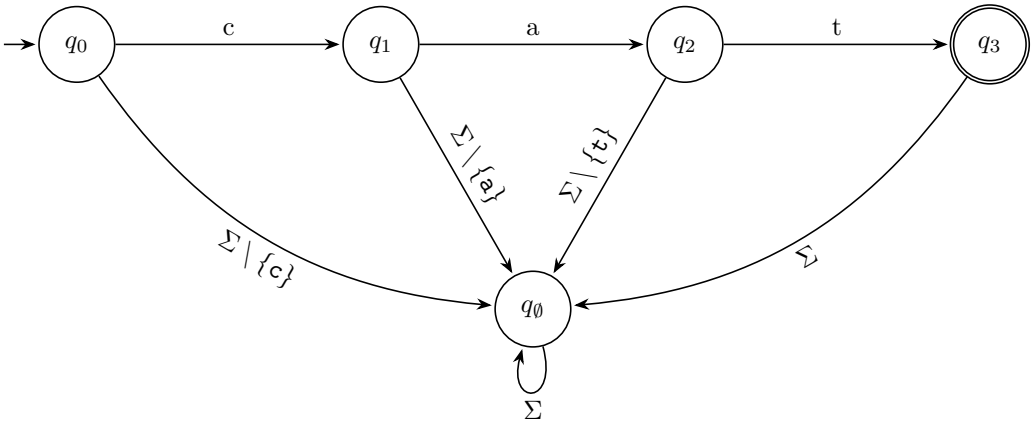
Definition. (Transition diagram) A *transition diagram* for a DFA, D, is a directed labelled graph with:

(i) The vertices being the states Q.
(ii) For each $(q, \sigma) \in Q \times \Sigma$, we draw a directed edge from q to $\delta(q, \sigma)$ and label this edge with σ. So each vertex has $|\Sigma|$ out-edges for every vertex.
(iii) Add an arrow from outside the diagram to q_0 labelled `START`.
(iv) Draw vertex as a circle or as a double circle if the state is an accept state.

If we have lots of edges from one state to another vertex, we can join them in a single edge with the label subset of Σ.

Consider the following diagram which shows a DFA which accepts the word 'cat' and nothing else.

$\Sigma = \{\mathtt{a}, \mathtt{b}, \ldots, \mathtt{z}, _\}$ ($_$ denotes a space)



Definition. (Transition table) For a DFA we can also show the corresponding information in a table. The columns are symbols in Σ and the rows are states $q_0, q_1, \ldots$ and the (i, j) entry is $\delta(q_i, \sigma_j)$.

2.2 Regular Languages

Definition. (Extended transition function) The *extended transition function* of a DFA, D, is

$$\hat{\delta} : Q \times \Sigma^* \to Q$$

defined inductively via

$$\hat{\delta}(q, \varepsilon) = q, \quad \hat{\delta}(q, \sigma_1) = \delta(q, \sigma_1), \quad \delta(q, \sigma_1 \cdots \sigma_k) = \delta(\hat{\delta}(q, \sigma_1 \ldots \sigma_{k-1}), \sigma_k)$$

for all $q \in Q$ and $\sigma_1, \ldots, \sigma_k \in \Sigma$.

So for any word $w \in \Sigma^*$, $\hat{\delta}(q_0, w)$ is the end state with input w.

Lemma. For all $1 \leq \ell \leq k$, we have

$$\hat{\delta}(q, \sigma_1 \ldots \sigma_k) = \hat{\delta}(\hat{\delta}(q, \sigma_1 \ldots \sigma_\ell), \sigma_{\ell+1} \ldots \sigma_k).$$

Proof. Induct on k. We take the base case where $k = \ell$ which follows from the definition. Otherwise,

$$\begin{aligned} \text{LHS} &= \delta(\hat{\delta}(q, \sigma_1 \ldots \sigma_{k-1}), \sigma_k) \\ &= \delta(\hat{\delta}(\hat{\delta}(q, \sigma_1 \ldots \sigma_\ell), \sigma_{\ell+1} \ldots \sigma_{k-1}), \sigma_k) = \text{RHS} \quad \square \end{aligned}$$

Definition. (Regular Language) Let D be a DFA. The language $\mathcal{L}(D)$ of D is the set of words accepted by D namely

$$\mathcal{L}(D) = \left\{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F \right\} \subseteq \Sigma^*$$

A language L is *regular* if there exists a DFA D such that $L = \mathcal{L}(D)$.

2.3 Non-determinism and ε-transitions

We now examine a seemingly more general construction where we can choose which state to move to (or equivalently move to many states simultaneously).

Definition. (Non-deterministic Finite-State Automaton) A *non-deterministic finite-state automaton* (NFA) is a structure

$$N = (Q, \Sigma, \delta, q_0, F)$$

just as a for a DFA, however δ is now a map $Q \times \Sigma \to \mathcal{P}(Q)$, to contain the idea of moving

to many states "at once".

An NFA as input $w = \sigma_1 \dots \sigma_k \in \Sigma^*$ while in state q_0, reads σ_1 then "moves to" all of the states in $\delta(q_0, \sigma_1)$. Then for all states $p \in \delta(q_0, \sigma_1)$, read σ_2 and move to $\sigma(p, \sigma_2)$. Continue to end of w to get to all end states $E_w \subseteq Q$. If $E_w \cap F \neq \emptyset$, then w is accepted, otherwise w is rejected.

Remark. Note that since $\emptyset \in \mathcal{P}(Q)$ we can have $\delta(p, \sigma) = \emptyset$. But what if we're currently in states $S \subseteq Q$ and to read σ_i only to find that $\bigcup_{p \in S} \delta(p, \sigma_i) = \emptyset$. Then we are in "no state" and we are stuck in NFA limbo until the end of the word, where the word is automatically rejected.

Given an NFA N, we can define the transition diagram and the transition table in the same way as for a DFA although the table now contains subsets of Σ and the diagram has multiple directed edges for a single letter in Σ.

Definition. (Extended transition function) For an NFA the *extended transition function* $\hat{\delta}$ of N is $\hat{\delta} : Q \times \Sigma^* \to \mathcal{P}(Q)$ defined by $\hat{\delta}(q, \varepsilon) = \{q\}$ for $q \in Q$, $\hat{\delta}(q, \sigma) = \delta(q, \sigma)$ for $q \in Q$ and $\sigma \in \Sigma$ and

$$\hat{\delta}(q, \sigma_1 \dots \sigma_k) = \bigcup_{p \in \hat{\delta}(q, \sigma_1 \dots \sigma_{k-1})} \delta(p, \sigma_k).$$

So for $w \in \Sigma^*$ we have that $\hat{\delta}(q_0, w)$ is the set of end states E_w.

We want to define a more general type of state machine. We want a more flexible type where we can change certain states without reading a symbol. We think about this as reading ε between the letters.

Definition. (ε-NFA) An *ε-NFA* is a structure $E = (Q, \Sigma, \delta, q_0, F)$ defined just as for an NFA except

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$$

where $\varepsilon \notin \Sigma$ but is acting like a symbol.

Definition. (ε-closure) For an ε-NFA, E, and a state $q \in Q$, we define the *ε-closure* $\text{ecl}(q)$ of q as the set of states reached from q by repeated ε-transitions.

That is define inductively, $S_0(q) = \{q\}$, $S_1 = \{q\} \cup \delta(q, \varepsilon)$ and $S_{i+1}(q) = S_i(q) \cup \bigcup_{r \in S_i(q)} \delta(r, \varepsilon)$. Then we can say that

$$\text{ecl}(q) = \bigcup_{i=0}^{\infty} S_i(q) = S_N(q)$$

for some N since we have a strictly increasing chain which balances at some point. We can also take the ε-closure of a subset $S \subseteq Q$ defined as $\text{ecl}(S)$, defined as the union of the ε-closure of each element in S. We have that the ε-closure moves in and out of unions.

Definition. (ε-closed) We say that $R \subseteq Q$ is *ε-closed* if $\text{ecl}(R) = R$.

Lemma. For any $S \subseteq Q$ we have that $\text{ecl}(S)$ is ε-closed.

Proof. Let $C = \text{ecl}(S)$. We need that $\text{ecl}(C) \subseteq C$. But $x \in \text{ecl}(C)$ means that $x \in \text{ecl}(r)$ for some $r \in C$, so there exists a finite sequence of ε-transitions from r to x. But $r \in \text{ecl}(S)$ so there exists some $s \in S$ with a sequence of ε-transitions from s to r. Putting these together we get that $x \in S_{m+n}(s) \subseteq \text{ecl}(s) \subseteq C$, where m and n are the lengths of the ε-transitions of the paths from r to x and r to s respectively. □

An ε-NFA has input $w = \sigma_1 \cdots \sigma_k \in \Sigma^*$. From q_0 it moves to all states in $\text{ecl}(q_0)$ and only then reads σ_1. For each $x \in \text{ecl}(q_0)$ it takes the set of the states $\delta(x, \sigma_1)$ and for each $y_x \in \delta(x, \sigma_1)$ it now goes to $\text{ecl}(y_x)$, so now in these states unioned over y_x and x. Continue with σ_2 and repeat, until we read the last letter σ_k and then w is accepted if any of these end states are in F, otherwise it is rejected.

Definition. (Extended transition function) For an ε-NFA, its *extended transition function* is a function

$$\hat{\delta} : Q \times \Sigma^* \to \mathcal{P}(Q)$$

defined inductvely on the length of the word $w = \sigma_1 \cdots \sigma_k$ via $\hat{\delta}(q, \varepsilon) = \text{ecl}(q)$.

$$\hat{\delta}(q, \sigma) = \text{ecl}\left(\bigcup_{p \in \text{ecl}(q)} \delta(p, \sigma) \right), \quad \hat{\delta}(q, \sigma_1 \ldots \sigma_k) = \text{ecl}\left(\bigcup_{p \in \hat{\delta}(q, \sigma_1 \ldots \sigma_{k-1})} \delta(p, \sigma_k) \right).$$

What is the relationship between these finite-state machines?

(i) Any DFA is a NFA (by changing the transition function to map to the singleton set insteaed).

(ii) Any NFA is an ε-NFA by defining $\delta(q, \varepsilon) = \emptyset$.

However all of these state machines actually define the same subset of languages called *regular languages*! We'll work up to a proof of that now.

Definition. (Language of NFAs and ε-NFAs) Given an NFA or a ε-NFA, E, it's language $\mathcal{L}(E)$ is given by

$$\mathcal{L}(E) = \left\{ w \in \Sigma^* : \hat{\delta}(q_0, w) \cap F \neq \emptyset \right\}.$$

2.4 Equivalence of DFAs and (ε-)NFAs

Any regular language is the language of an (ε-)NFA which is fairly simple to see. We want ot show the converse using the *subset construction*

First suppose we have an NFA, N. We produce a DFA D on the same alphabet Σ such that $\mathcal{L}(N) = \mathcal{L}(D)$.

Definition. (Subset construction) For the NFA $N = (Q, \Sigma, \delta, q_0, F)$, the *subset construc-*

tion for N produces the DFA

$$D = (\mathcal{P}(Q), \Sigma, \Delta, \{q_0\}, \mathcal{F})$$

where $\mathcal{F} = \{S \subseteq Q : S \cap F \neq \emptyset\}$ and for $S \subseteq Q, \sigma \in \Sigma$ we have that

$$\Delta(S, \sigma) = \bigcup_{p \in S} \delta(p, \sigma) \in \mathcal{P}(Q).$$

Theorem. For the NFA N and DFA D obtained from the subset construction, let $\hat{\delta}$ and $\hat{\Delta}$ to be their respective extended transition functions. Then for all $w \in \Sigma^*$ we have that

$$\hat{\Delta}(\{q_0\}, w) = \hat{\delta}(q_0, w).$$

Proof. Induct on word length $k = |w|$. The base case $k = 0$ is $w = \varepsilon$ then the LHS $= \{q_0\} =$ RHS. Now the inductive hypothesis

$$\hat{\Delta}(\{q_0\}, \sigma_1 \dots \sigma_{k-1}) = \hat{\delta}(q_0, \sigma_1 \dots \sigma_{k-1}).$$

Then for $w = \sigma_1 \dots \sigma_k$, we have that

$$\begin{aligned}\hat{\delta}(q_0, w) = \bigcup_{p \in \hat{\delta}(q_0, \sigma_1 \dots \sigma_{k-1})} \delta(p, \sigma_k) &= \Delta(\hat{\delta}(q_0, \sigma \dots \sigma_{k-1}), \sigma_k) \\ &= \Delta(\hat{\Delta}(\{q_0\}, \sigma_1 \dots \sigma_{k-1}), \sigma_k) = \hat{\Delta}(\{q_0\}, \sigma_1 \dots \sigma_k) \quad \square\end{aligned}$$

Corollary. A language L is accepted by some DFA if and only if it accepted by some NFA.

Proof. Forwared direction is obvious. In the converse, suppose we're given an NFA N, form a DFA D from the subset construction. Then w is accepted by N if and only if $\hat{\Delta}(\{q_0\}, w) \in \mathcal{F}$ by the previous theorem. $\square$

Definition. (Subset construciton) For the ε-NFA, $E = (Q, \Sigma, \delta, q_0, F)$ the *subset construction* with ε-transitions produces the DFA $D(\mathcal{P}(Q), \Sigma, \Delta, \text{ecl}(\{q_0\}), \mathcal{F})$ where $\mathcal{F} = \{S \subseteq Q | S \cap F \neq \emptyset\}$ as before and for $S \subseteq Q, \sigma \in \Sigma$ we have that

$$\Delta(S, \sigma) = \text{ecl}\left(\bigcup_{p \in S} \delta(p, \sigma)\right) \in \mathcal{P}(Q).$$

Remark. This can be put into words as the subsets of states reached from S by following an edge labelled σ then some edges labelled ε.

As before we have a theorem proving this constructions defines the same language.

Theorem. For an ε-NFA, E, and DFA, D, obtained from the subset construction, we have that

$$\hat{\Delta}(\text{ecl}(\{q_0\}), w) = \hat{\delta}(q_0, w).$$

Proof. As in the previous theorem, the base case for $w \in \varepsilon$ gives $\text{ecl}(\{q_0\})$. Then the inductive steps is similar to the previous proof with the ecl places and handled with appropiately. □

Corollary. A language L is accepted by some DFA if and only if it is accepted by some NFA if and only if it is accepted by some ε-NFA.

2.5 Regular Expressions

We know now that DFAs and (ε-)NFAs defnie the same languages. We knwo want to give an algebraic or symbolic description of a regular language: that of a regular expression. Then we will show that DFAs $\implies$ Regular Expressions $\implies$ ε-NFAs, which will show they're all equivalent.

Definition. (Creating new languages from old)

(i) The *union* of languages L, M over Σ_L, Σ_M is the language $L \cup M$ over $\Sigma_L \cup \Sigma_M$.

(ii) The *concatenation* LM of languages L, M is the words $w = uv$ where $\{(u, v) \in L \times M\} \subseteq (\Sigma_L \cup \Sigma_M)^*$.

(iii) The *closure* L^* of a language L is a set of all words formed by concatenating an arbitrary number of wordes in L so

$$L^* = \bigcup_{n \geq 0} L^n$$

including the empty word.

Remark. For $n \geq 2$ we can have $L \not\subseteq L^n$ if $\varepsilon \notin L$, but not if $\varepsilon \in L$ since we can use the empty word to pad the word out. However certainly $L \subseteq L^*$.

Definition. (Regular Expression) A *regular expression* R is a finite string of symbols denoting a language $\mathcal{L}(R)$ over some alphabet, build up from the following inductive rules:

(i) Both ε and $\emptyset$ are regular expressions with $\mathcal{L}(\varepsilon) = \{\varepsilon\}$ and $\mathcal{L}(\emptyset\} = \emptyset$.

(ii) If a is a symbol then a is a regular expression with $\mathcal{L}(a) = \{a\}$.

(iii) If E and F are regular expressions then $E + F$ is a regular expression denoting union so $\mathcal{L}(E + F) = \mathcal{L}(E) \cup \mathcal{L}(F)$.

(iv) EF is a regular expression denoting concatenation, so $\mathcal{L}(EF) = \mathcal{L}(E)\mathcal{L}(F)$.

(v) E^* is a regular expression denoting the closure of E so $\mathcal{L}(E^*) = \mathcal{L}(E)^*$.

(vi) (E) is a regular expression with the same language of E and brackets are used to avoid ambiguity and enforce order of operations.

Definition. (Order of operations) The order of precendence for regular expressiosn is brackets, then Kleene star, then concatenation, then addition.

Example. For example the regular expression $01^* + 1$ defines the language accepting 1 or anything of the form 0(any number of 1s).

2.5.1 Regular expressions define regular languages

Theorem. For each regular expression R, there exists an ε-NFA, E, such that $\mathcal{L}(E) = \mathcal{L}(R)$.

Proof. We build E up by induction. Our ε-NFA will be constructed such that

(i) There is a unique accepting state;

(ii) There are no directed edges into the start state accept `START`;

(iii) No directed edges out of the accepting state.